

Guía del Código: WebXR Drag & Depth Occlusion

Trofeo 3D en Realidad Aumentada con Arrastre y Profundidad

1. ¿Qué hace este proyecto?

Este código muestra un **trofeo 3D** en tu sala real usando la cámara del teléfono. Puedes:

- **Ver el trofeo** flotando y girando en tu espacio físico
- **Agarrarlo con el dedo** y moverlo por la sala (**DRAG**)
- **Ver cómo se oculta** detrás de mesas, sillas u objetos reales (**profundidad**)

2. Concepto: DRAG (Arrastre)

¿Qué es?

DRAG significa "arrastrar". En este contexto, es la acción de tocar un objeto virtual con el dedo, mantener presionado y moverlo por la pantalla como si fuera un objeto físico.

¿Cómo funciona en el código?

El sistema usa un **rayo invisible** que sale de tu dedo hacia la escena 3D. Cuando ese rayo toca el objeto, el sistema lo "adopta" y lo mueve contigo.

Evento	Función del código	Qué siente el usuario
Tocar el objeto	<code>onSelectStart</code>	El objeto se ilumina
Mantener y mover	<code>controller.attach(object)</code>	El objeto sigue el dedo
Soltar	<code>onSelectEnd</code>	El objeto queda en su nuevo lugar

El truco del contenedor invisible

El código no hace drag directamente sobre el trofeo. En su lugar, usa un **cubo invisible** que envuelve al trofeo:

Cubo invisible (zona de toque)
└─ Trofeo 3D (visual)

¿Por qué? El trofeo tiene una forma compleja con muchas caras. Calcular colisiones contra todas esas caras en cada frame es lento. Un cubo simple es mucho más rápido de intersectar.

```
const geometry = new THREE.BoxGeometry(0.4, 0.4, 0.4);
const hitMaterial = new THREE.MeshBasicMaterial({
  transparent: true,
  opacity: 0.0 // ← INVISIBLE
});
```

El cubo no se ve (`opacity: 0.0`), pero sí responde al toque. Es como un "campo de fuerza" alrededor del trofeo.

Candado de rotación

Cuando sueltas el trofeo, el código fuerza una rotación específica:

```
object.rotation.set(0, 0, 0);
```

Esto garantiza que la **base del trofeo siempre mire hacia abajo**, como si estuviera posado sobre una mesa. No importa cómo lo hayas inclinado al arrastrarlo: al soltar, se endereza.

3. Concepto: Profundidad (Depth Sensing)

¿Qué es?

La **profundidad** es la capacidad del teléfono de medir qué tan lejos están los objetos reales de tu sala. Usa la cámara de profundidad (si el dispositivo la tiene) para crear un **mapa de distancias**.

¿Cómo funciona?

1. La cámara IR (infrarrojo) escanea la sala
2. Genera una imagen donde cada píxel representa una distancia en metros
3. El código compara: ¿el trofeo virtual está más cerca o más lejos que la mesa real?

Oclusión: el efecto mágico

Oclusión significa que un objeto real tapa a uno virtual. Sin profundidad, el trofeo siempre se vería "encima" de todo. Con profundidad:

Situación	Sin profundidad	Con profundidad
Mesa delante del trofeo	Trofeo atraviesa la mesa	Trofeo se oculta detrás
Mano delante del trofeo	Trofeo flota sobre la mano	Trofeo desaparece tras la mano
Pared detrás del trofeo	Trofeo sobresale de la pared	Trofeo se corta correctamente

En el código

La profundidad se solicita al iniciar WebXR:

```
document.body.appendChild(XRButton.createButton(renderer, {
  'optionalFeatures': ['depth-sensing'],
  'depthSensing': {
    'usagePreference': ['gpu-optimized'],
    'dataFormatPreference': ['luminance-alpha']
  }
}));
```

Y se verifica en cada frame:

```
if (renderer.xr.hasDepthSensing() && !isDepthSupplied) {
  const depthTexture = renderer.xr.getDepthTexture();
  // ... pasar al shader
}
```

Limitación actual

En esta versión, el trofeo usa su **material original** del modelo GLB. El shader de profundidad está preparado en el código pero no se aplica directamente al trofeo. La oclusión funciona a nivel de sistema WebXR, no por shader custom.

4. Estructura del Código

Función `init()` — La preparación

Crea todo lo necesario antes de mostrar algo:

Sección	Qué crea
Escena y cámara	El mundo 3D y el punto de vista
Luces	Iluminación ambiental y direccional
Suelo	Plano invisible que recibe sombras
Cubo contenedor	Zona de toque invisible para drag
Carga del trofeo	Lee el archivo <code>Trophy.glb</code>
Renderer	Motor de dibujo con WebXR activado
Controlador	El "dedo virtual" que intersecta objetos

Función `animate()` — El loop infinito

```
function animate() {  
    renderer.setAnimationLoop(render);  
}
```

Esto dice: "cada vez que la pantalla esté lista para un nuevo frame, ejecuta `render()`". Es un ciclo que nunca termina, típico de videojuegos y aplicaciones 3D.

Función `render()` — Cada frame

Se ejecuta 60 veces por segundo (o más). En cada llamada:

1. **Calcula el tiempo transcurrido** para la animación
 2. **Rota y levita el trofeo** (animación continua)
 3. **Verifica si hay profundidad** disponible
 4. **Limpia objetos previamente intersectados**
 5. **Busca nuevas intersecciones** con el dedo/controlador
 6. **Actualiza controles** y dibuja la escena
-

5. Animación del Trofeo

El trofeo tiene dos movimientos constantes:

Rotación sobre el eje Y

```
model.rotation.y = elapsedTime * 1.5;
```

Gira como una peonza. `elapsedTime` es el tiempo en segundos desde que inició la app. Multiplicar por `1.5` controla la velocidad: a mayor número, gira más rápido.

Levitación (flotación)

```
model.position.y = Math.sin(elapsedTime * 2.5) * 0.03;
```

Usa la función seno (`Math.sin`) para crear un movimiento ondulatorio suave. El trofeo sube y baja `0.03` metros (3 centímetros) de forma cíclica.

El modelo puede traer animación embebida del Unity Asset Store (rotación y flotación). El código en `render()` también aplica transformaciones. Ambas coexisten sin conflicto en este caso.

6. Reglas para los Estudiantes

✓ Sí puedes hacer

Acción	Cómo
Reemplazar el modelo	Sube tu <code>.glb</code> a <code>models/gltf/</code> y cambia el nombre en el código
Cambiar escala	Modifica <code>trophyModel.scale.set(x, x, x)</code>
Cambiar velocidad de giro	Modifica el <code>* 1.5</code> en <code>model.rotation.y</code>
Cambiar altura de flotación	Modifica el <code>* 0.03</code> en <code>model.position.y</code>
Cambiar posición inicial	Modifica <code>object.position.set(0, 1.0, -1.5)</code>
Ajustar tamaño del cubo	Modifica <code>BoxGeometry(ancho, alto, profundo)</code>

✗ NO puedes hacer

Acción	Por qué falla
Modelos con huesos (rig)	El skinning se descompone al hacer drag
Modelos con animaciones internas	Chocan con la rotación/levitación por código
Múltiples objetos con drag	El código solo maneja un contenedor
Poner el modelo fuera de la carpeta <code>models/gltf/</code>	La ruta está hardcodeda
Usar formatos que no sean <code>.glb</code>	El loader solo lee GLTF/GLB

7. Cómo Reemplazar el Modelo

Paso 1: Prepara tu modelo en Unity

- Modelo low poly (pocos polígonos)
- Sin huesos, sin animaciones, sin scripts C#
- Material básico con o sin textura
- Escala 1:1:1 en Unity

Paso 2: Exporta con Unity Recorder

- 1.Window → General → Recorder → Recorder Window
- 2.Selecciona formato de salida (FBX estático)
- 3.Graha un solo frame
- 4.Convierte el FBX a GLB usando Blender o gltf.report

Paso 3: Sube a tu repositorio

Coloca el archivo en:

`models/gltf/TuModelo.glb`

Paso 4: Cambia el código

Busca esta línea:

```
loader.load('./models/gltf/Trophy.glb', function (gltf) {
```

Cámbiala por:

```
loader.load('./models/gltf/TuModelo.glb', function (gltf) {
```

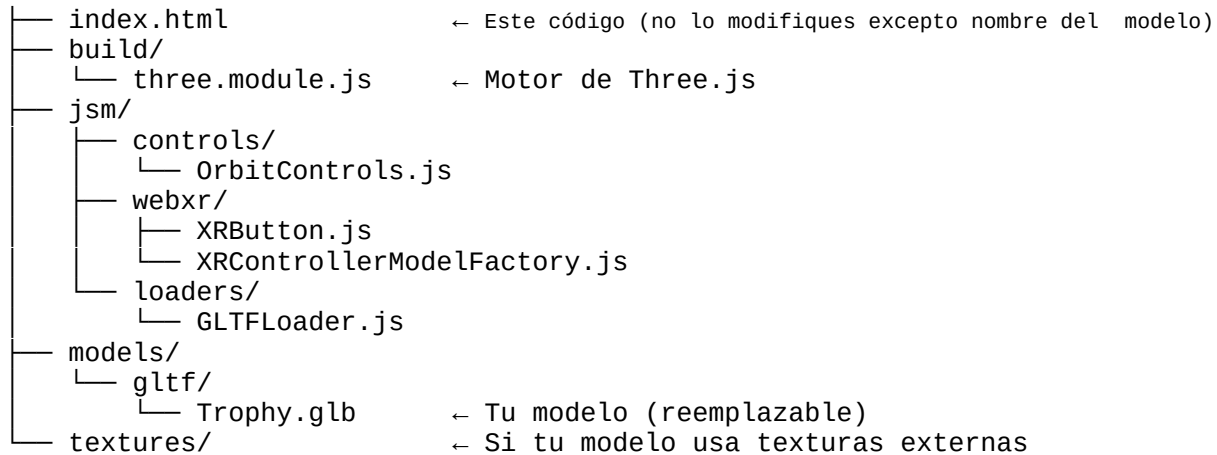
Paso 5: Ajusta la escala

Prueba valores hasta que se vea bien:

```
trophyModel.scale.set(1.0, 1.0, 1.0); // Prueba 0.5, 1.0, 1.5, 2.0...
```

8. Estructura de Archivos del Proyecto

Lab12/



9. Glosario de Términos

Término	Significado
WebXR	API de JavaScript para realidad virtual y aumentada en navegadores
GLB	Formato de archivo 3D que empaqueta modelo, texturas y materiales en uno solo
Shader	Programa que corre en la GPU para calcular colores y efectos visuales
Raycaster	Sistema que lanza un rayo invisible para detectar colisiones en 3D
Depth Sensing	Tecnología que mide distancias a objetos reales usando cámara de profundidad
Occlusion	Efecto visual donde un objeto tapa a otro según su profundidad
Frame	Una imagen individual en una secuencia de animación (60 fps = 60 frames por segundo)
Mesh	Objeto 3D compuesto por vértices, aristas y caras
Material	Propiedades visuales de una superficie: color, brillo, transparencia
Texture	Imagen 2D aplicada sobre una superficie 3D

10. Resumen Visual del Flujo

```
Usuario toca la pantalla
↓
Raycaster lanza rayo desde el dedo
↓
¿El rayo intersecta el cubo invisible?
↓
Sí → Cubo se ilumina (opacity: 0.3)
↓
Usuario mantiene presionado
↓
controller.attach(cubo) → Cubo sigue el dedo
↓
Trofeo (hijo del cubo) se mueve junto con él
↓
Usuario suelta
↓
group.attach(cubo) → Cubo vuelve al grupo principal
↓
object.rotation.set(0,0,0) → Base enderezada hacia abajo
↓
Trofeo sigue girando y flotando (animación continua)
```

11. Código QR

